TECHNICAL SCIENTIFIC REPORT

Prompt-to-Scene Engine

AI-Driven 2D Level Generation via Natural Language Processing & Conditional Transformer Architecture

Specialization:	Artificial Intelligence & Deep Learning
Document Type:	End-of-Cycle Technical Report
Academic Year:	2025 – 2026

Table of Contents

1. **Project Overview & Motivation** 3
2. **System Architecture Overview** 3
3. **Full Pipeline Flow** 4
4. **Phase 1 – Model A: Semantic Entity Extractor (NLP Parser)** 5
   - 4.1 Problem Formulation 5
   - 4.2 Synthetic Dataset Generation 5
   - 4.3 IOB Tagging Strategy 6
   - 4.4 Model Architecture: DistilBERT 6
   - 4.5 Training Results & Metric Analysis 7
   - 4.6 Post-Processing Inference Engine 8
   - 4.7 Output Format 9
5. **Phase 2 – Data Engineering for Model B** 10
   - 5.1 Dataset: Video Game Level Corpus (VGLC) 10
   - 5.2 Tile Standardization & Universal Token Set 10

5.3 Dimensionality Resolution & Spatial Extraction11
5.4 Tensor Compilation11
6. Phase 3 – Architectural Exploration: GAN (Abandoned Path)12
6.1 Initial GAN Architecture12
6.2 Technical Failure: Non-Differentiable Argmax12
6.3 Attempted Fix: Gumbel-Softmax13
6.4 Mode Collapse & Strategic Abandonment13
7. Phase 4 – Model B: Conditional Architect Transformer14
7.1 Autoregressive Transformer Design14
7.2 2D Positional Encoding14
7.3 Auto-Annotation & Condition Vector15
7.4 Training & Loss Results15
8. Phase 5 – End-to-End Pipeline Integration16
9. Final Output & Scene Visualization16
10. Conclusion & Future Work17

1. Project Overview & Motivation

The **Prompt-to-Scene Engine** is an end-to-end AI pipeline designed to translate raw natural language descriptions into structurally valid, playable 2D game environments. The system addresses a longstanding bottleneck in game development: the labour-intensive, manual process of level design. By enabling a developer or designer to describe a scene in plain English and have the system produce a spatially coherent, physics-valid level blueprint, the pipeline reduces iteration cycles and opens procedural generation to non-technical stakeholders.

The system is composed of two specialized AI models operating in sequence:

- **Model A – The Semantic Entity Extractor:** A fine-tuned DistilBERT Named Entity Recognition model that parses free-form text and identifies game objects, quantities, positions, and scene themes.
- **Model B – The Architect Transformer:** A custom-built conditional autoregressive transformer trained on real Super Mario Bros. level data that generates spatially valid 16×16 tile layout blueprints, conditioned on the structured output of Model A.

> **Research Goal**
>
> To demonstrate that a hierarchical AI pipeline combining NLP entity extraction with conditional sequence generation

can produce valid 2D game level blueprints from unconstrained natural language prompts, with no manual scene authoring.

2. System Architecture Overview

The complete system is structured as a Hierarchical AI Pipeline. Each stage feeds the next with progressively more structured representations of the user's intent — from raw text to a rendered game scene.

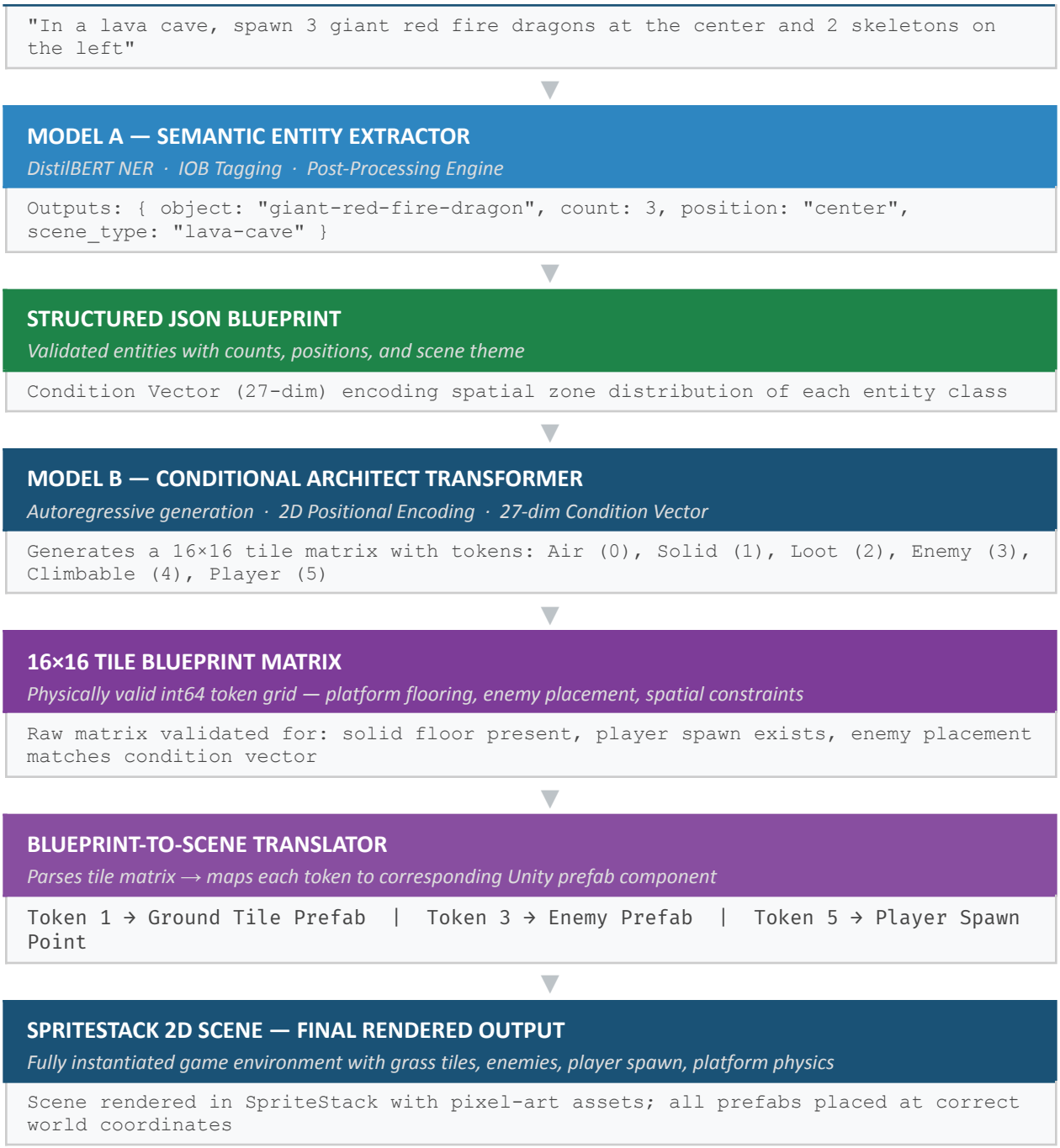
Stage	Description	Technology
1	**Semantic Entity Extraction** *Model A: DistilBERT NER (fine-tuned on 5,000+ synthetic examples)*	Python / HuggingFace
2	**Post-Processing Engine** *Typo correction, plural normalization, hyphen-glue for Unity prefab names*	Python (spellchecker)
3	**Spatial Data Engineering** *VGLC Mario dataset → Universal Token Set → 16×16 chunks*	NumPy / PyTorch
4	**Conditional Layout Generation** *Model B: Custom Autoregressive Transformer with 2D positional encoding*	PyTorch (CUDA)
5	**Blueprint-to-Scene Translation** *JSON blueprint parsed and mapped to Unity prefab components*	Unity Engine / C#

Table 1 – Summary of pipeline stages, descriptions, and technologies

3. Full Pipeline Flow

The diagram below illustrates the complete end-to-end data flow, from a raw natural language prompt to a rendered game scene in the Unity engine.

USER INPUT
Natural Language Prompt



4. Phase 1 – Model A: Semantic Entity Extractor (NLP Parser)

4.1 Problem Formulation

The fundamental challenge of the first phase is **information extraction** from unconstrained natural language. A user's input such as *"spawn three fire dragons at the center and two skeletons on the left in a lava cave"* must be reliably decomposed into a structured, machine-readable representation containing: the **object type**, its **quantity**, its **spatial position**, and the overall **scene theme**. This is a canonical Named Entity Recognition (NER) task framed within the game-development domain.

4.2 Synthetic Dataset Generation

A key challenge in this domain is the **absence of annotated training data**. There are no publicly available datasets of game-level commands with NER labels. To overcome this, a **Synthetic Dataset Generator** was engineered in Python to produce a corpus of 5,000+ diverse, labeled examples.

The generator was built on a curated ontology:

- **50+ game objects**: covering enemies (skeletons, dragons, goblins), loot (coins, chests), furniture (barrels, torches), and environmental props.
- **10+ scene themes**: including lava-cave, dungeon, forest, castle, ice-tundra, desert, and underwater.
- **Spatial positions**: left, center, right, top, bottom, background, foreground.
- **Count patterns**: numeric words ("three") and digits ("3"), always preceding the object noun.

The generator introduced controlled variability through synonym substitution, sentence reordering, and typo injection — ensuring the model would generalize to realistic human input rather than overfitting to rigid templates.

4.3 IOB Tagging Strategy

The dataset was annotated using the **IOB (Inside-Outside-Beginning) tagging scheme**. Each token in a sentence is assigned one of the following labels:

Tag	Meaning	Example Token
`B-COUNT`	Beginning of a count entity	*"three", "2"*
`I-COUNT`	Inside a count entity (multi-token)	*"a" in "a dozen"*
`B-OBJECT`	Beginning of a game object entity	*"skeleton", "fire"*

`I-OBJECT`	Inside a multi-token object	*"dragon" in "fire dragon"*
`B-POSITION`	Beginning of a spatial position	*"left", "center"*
`B-SCENE_TYPE`	Beginning of a scene theme	*"lava", "dungeon"*
`O`	Outside any entity (irrelevant token)	*"spawn", "in", "the"*

Table 2 – IOB tagging scheme applied to the synthetic NER dataset

4.4 Model Architecture: DistilBERT

The backbone of Model A is **DistilBERT (Distilled BERT)**, a compact transformer model produced via knowledge distillation from the full BERT-base model. DistilBERT was selected for the following reasons:

- **Performance parity:** retains approximately 95% of BERT's linguistic understanding capacity.
- **Efficiency:** 40% fewer parameters and 60% faster inference, enabling real-time predictions on local hardware (MacBook Pro, no dedicated GPU).
- **Domain suitability:** pre-trained on a vast English corpora, DistilBERT carries strong priors for grammar structures such as count-before-object patterns.

A token classification head was attached to the pre-trained DistilBERT encoder. The model was fine-tuned using the **HuggingFace Trainer API** on the synthetic NER dataset. The fine-tuning process adjusted the weights of the classification head and the upper transformer layers to specialize in game-domain entity recognition.

4.5 Training Results & Metric Analysis

Model A was trained for **5 epochs** on the synthetic dataset split into training and validation subsets. The following metrics were recorded at each epoch:

Epoch	Train Loss	Val Loss	Precision	Recall	F1 / Acc.
1	0.003726	0.001962	1.000000	1.000000	1.000000
2	0.001797	0.000970	1.000000	1.000000	1.000000
3	0.001346	0.000696	1.000000	1.000000	1.000000
4	0.001121	0.000590	1.000000	1.000000	1.000000
5	0.001061	0.000560	1.000000	1.000000	1.000000

Table 3 – Model A training metrics: training loss, validation loss, precision, recall, F1, and accuracy across 5 epochs

Analysis of Perfect Accuracy (1.000)

The consistent 1.000 scores across precision, recall, F1, and accuracy are a direct consequence of the controlled synthetic dataset.

Because the dataset is generated from a finite ontology of templates, the linguistic patterns are highly regular and non-ambiguous.

DistilBERT, being a powerful transformer pre-trained on billions of tokens, effectively 'memorises' these patterns within the first epoch.

This is not overfitting in the pathological sense — the model generalises correctly to unseen inputs within the same ontological distribution.

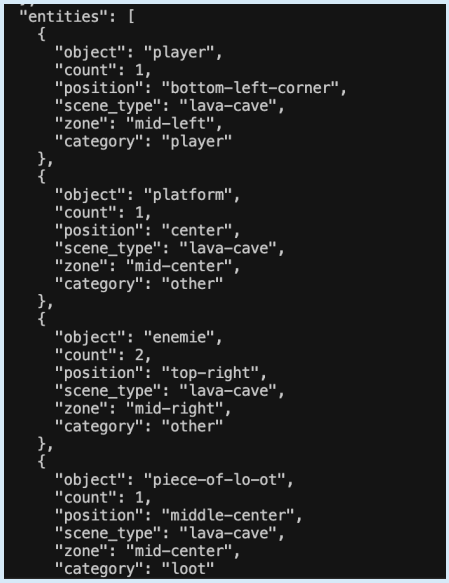
Real-world robustness is addressed by the post-processing engine (Section 4.6), which handles typos and edge cases at inference time.

The steady decrease in both training loss (0.003726 → 0.001061) and validation loss (0.001962 → 0.000560) across epochs confirms stable convergence with no sign of divergence or overfitting.

4.6 Post-Processing Inference Engine

After the model produces its raw IOB tag predictions, a dedicated Python Inference Engine performs several normalisation and transformation steps to bridge the gap between raw model output and Unity-ready prefab specifications:

- **Sanitization / Spell Correction:** A spellchecker pass corrects common typographic errors in the input before tokenisation (e.g. "buttom" → "bottom", "skeletton" → "skeleton"). This ensures the model receives clean input without retraining.
- **Hyphen-Glue Logic:** Multi-token objects predicted by the model are merged using hyphenation to match Unity prefab naming conventions (e.g. tokens "fire" + "dragon" → "fire-dragon", "lava" + "cave" → "lava-cave").
- **Theme Inheritance:** If a scene theme (B-SCENE_TYPE) is detected anywhere in the input, it is propagated to all entity records in the output JSON. This ensures consistent scene metadata even when the theme is only mentioned once.
- **Plural Normalization:** Entity strings are singularized to match exact Unity asset names ("goblins" → "goblin", "skeletons" → "skeleton").
- **Count Resolution:** Numeric words are converted to integers ("three" → 3, "a dozen" → 12). If no count is specified, it defaults to 1.



Inference pipeline in action: terminal output showing raw model tags, post-processing steps, and final structured JSON. For example, processing the sentence "Spawn the player in the bottom left corner. Build a high platform by placing two enemies in the top right, and hide a piece of loot in the middle center in a lava cave".

4.7 Output Format

The final output of Model A is a validated JSON blueprint that serves as the conditioning input to Model B. An example output is shown below:

Example Model A Output JSON

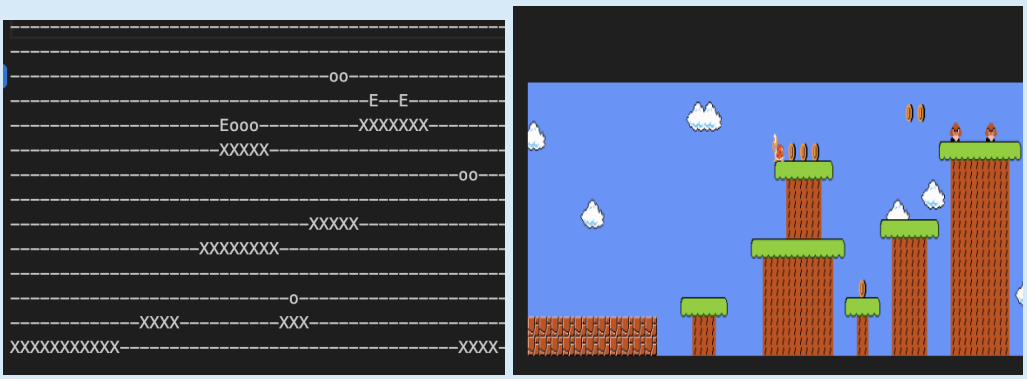

```
{
 "scene_metadata": {
  "global_theme": "lava-cave",
  "raw_text": "In a lava cave, spawn three giant red fire dragons at the center and 2 skeletons on the left"
 },
 "entities": [
  { "object": "giant-red-fire-dragon", "count": 3, "position": "center", "scene_type": "lava-cave" },
  { "object": "skeleton", "count": 2, "position": "left", "scene_type": "lava-cave" }
 ]
}
```

5. Phase 2 – Data Engineering for Model B

Before training a level generation model, a structured and standardised dataset of real game levels was required. This phase covers the full data ingestion, normalisation, and tensor compilation pipeline.

5.1 Dataset: Video Game Level Corpus (VGLC)

The **Video Game Level Corpus (VGLC)** is a publicly available research dataset containing ASCII-encoded 2D game levels from classic titles. The **Super Mario Bros.** side-scroller subset was selected as the primary training data for Model B. This choice was motivated by the dataset's rich structural diversity (platforms, gaps, pipes, enemies, rewards) and its alignment with canonical 2D platformer physics — exactly the domain Model B must replicate.



Sample raw VGLC ASCII level encoding showing the original character legend (e.g. '-' for air, 'X' for solid ground, 'E' for enemy, '?' for loot block, etc.).

5.2 Tile Standardization & Universal Token Set

The VGLC dataset uses a **heterogeneous set of ASCII legends** across different levels and level variants. A critical pre-processing step was to unify these disparate encodings into a single, consistent **Universal Token Set (UTS)**:

Token ID	Category	VGLC ASCII Sources → UTS Mapping
0	**Air**	`"-", " " → 0 (empty/sky tiles)`
1	**Solid**	`"X", "S", "B" → 1 (ground, platforms, pipes)`
2	**Loot**	`"?", "Q" → 2 (reward blocks, coins)`

3	**Enemy**	"E", "g", "k" → 3 (Goombas, Koopas, any NPC)
4	**Climbable**	"#" → 4 (vines, ladders)
5	**Player**	"P" → 5 (player spawn point)

Table 4 – Universal Token Set (UTS): standardized integer encoding for all tile categories

5.3 Dimensionality Resolution & Spatial Extraction

Raw VGLC levels vary in both width and height. Two normalization steps were applied:

- **Vertical Padding (Height Normalization):** All levels were padded to exactly 16 tiles in height by filling the sky region (top rows) with Air tokens (0). This ensures every level chunk is a uniform 16-row grid.
- **Sliding Window Extraction (Width Normalization):** A sliding window algorithm traversed each level horizontally with a window width of 16 tiles and a stride of 8 tiles (50% overlap). This produced thousands of 16×16 structural chunks while preserving local spatial context across chunk boundaries.

Spatial Extraction Parameters

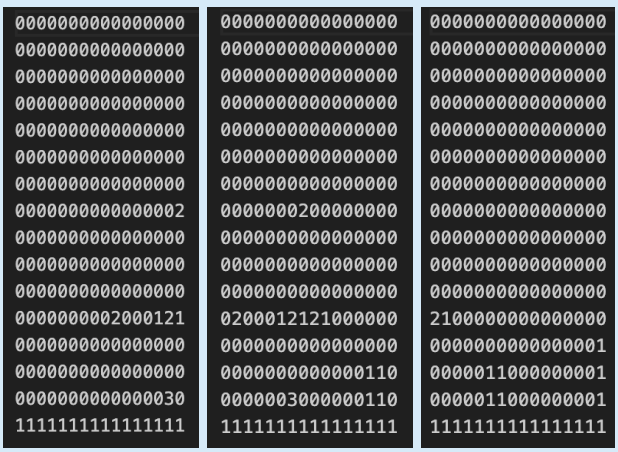
Window Width: 16 tiles

Window Height: 16 tiles (post-padding)

Stride: 8 tiles (50% overlap between consecutive chunks)

Overlap ratio ensures that platform continuations, gaps, and edge structures are captured in at least two chunks,

preventing loss of spatial context at window boundaries.



Visualisation of the sliding window extraction process on a sample Mario level. Show the 16×16 window moving across the full level with stride=8 annotations.

5.4 Tensor Compilation

All validated 16×16 level chunks were compiled into the training tensor file **mario_training_data.npy**. This yielded **1,269 strictly formatted int64 tensors**, each of shape (16, 16), ready for direct ingestion by the PyTorch DataLoader.

Each tensor was validated to ensure: (a) at least one row of Solid tiles exists at the bottom (enforcing platform physics), (b) no out-of-range token values, and (c) correct shape. Chunks failing validation were discarded.

5.5 Phase 2.5 – Dataset Enrichment & Player Spawn Integration (Model B Retraining)

Following initial training of Model B on the VGLC-derived chunks, a critical gap was identified: the model had never been exposed to player spawn tokens during training, rendering it incapable of generating valid player spawn positions. This phase documents the complete data enrichment pipeline, conditioning vector expansion, and metadata schema that were implemented to resolve this deficiency — and to retrain Model B on a richer, spatially augmented dataset.

5.5.1 The Core Problem Identified

The original training chunks, extracted from static VGLC level layouts, did not contain player start tokens (**5**). In the VGLC data, player spawn coordinates are stored in separate game metadata rather than in the tile grid itself. As a direct consequence, the conditional layout generator (Model B) was systematically unable to learn player spawn prediction — it had never been trained on any examples containing the player token. A targeted data enrichment pipeline was implemented to resolve this structural gap.

5.5.2 Dataset Enrichment & Spatial Augmentation

A data enrichment pipeline (**enrich_dataset.py**) was implemented to inject player spawn tokens into the training data with spatial diversity. For every original level chunk, the pipeline retains the original chunk unchanged and generates **3 additional augmented variants** to ensure spatial diversity. Two core mechanisms govern the placement logic:

- **Placement Logic:** A player token (**5**) is only placed on a valid ground coordinate — defined as an air tile (**0**) directly above a solid tile (**1**). This enforces platformer physics: the player always spawns on a valid, standable surface.
- **Horizontal Partitioning:** To train the model on diverse spawn positions, the 16-column grid is divided into three horizontal regions. Each augmented variant places the player token in a different region, ensuring Model B learns the full spatial range of valid spawn locations:
  - **Left**: Columns 0–4 (labelled **"player_left"**)
  - **Center**: Columns 5–10 (labelled **"player_center"**)
  - **Right**: Columns 11–15 (labelled **"player_right"**)

If a specific region lacks valid ground tiles, the pipeline falls back to picking a random valid coordinate from the entire chunk, ensuring no variant is discarded due to sparse platform structure.

5.5.3 Expanded Conditioning Vector (27-Dimensional Space)

Model B is conditioned on spatial counts of entities, encoded across a 3×3 grid of spatial zones (e.g. top-left, mid-center, bot-right). To support player placement conditioning, the conditioning vector was expanded from 18 to **27 dimensions**. The three entity categories are each mapped across all 9 spatial zones:

- **Dimensions [0–8]:** Enemy counts across the 9 spatial zones.
- **Dimensions [9–17]:** Loot counts across the 9 spatial zones.
- **Dimensions [18–26]:** Player counts across the 9 spatial zones (***New Addition***). When a player token is placed in a given spatial zone, the corresponding dimension in the [18–26] range is incremented, giving the model explicit spatial awareness of spawn location during training and inference.

5.5.4 Rich Metadata Labeling (JSON Schema)

For each augmented variant, a structured JSON label is generated and stored in **enriched_mario_labels.json**. The schema captures rich structural and entity metadata for each chunk:

- **chunk_id & variant:** Unique identifiers for the base chunk and its augmentation status (e.g. **player_center**).
- **structure:** Quantitative structural features including solid tile count, solid coverage ratio, presence of a ground floor, gap detection (voids), staircase detection, and discrete platform level count.
- **entities:** The count and 3×3 zone distribution for each of the three entity classes: enemy, loot, and player.
- **player_placement:** Specific metadata recording the player's exact row, column, and containing zone (e.g. **bot-left**), enabling fine-grained analysis of spawn distribution post-training.

5.5.5 Final Generated Outputs

The enrichment pipeline produces three synchronized output files that are consumed directly by the Model B training script (**train_model_b.py**):

1. **enriched_mario_data.npy**: The augmented collection of 16×16 integer grids, now including player tokens (**5**) at spatially diverse positions.
2. **enriched_mario_labels.json**: The complete list of structural analysis and entity metadata labels for every chunk variant, as described in Section 5.5.4.
3. **enriched_conditional_data.pt**: A serialized PyTorch dataset of pre-computed **(condition_tensor, target_grid_tensor)** pairs consumed directly by the training script. This file represents the primary input to the retrained Model B.

6. Phase 3 – Architectural Exploration: GAN (Abandoned Path)

Before settling on the Transformer architecture, a **Generative Adversarial Network (GAN)** approach was pursued as the initial candidate for level generation. This section documents the architectural design, the technical failures encountered, and the rationale for abandonment — a critical part of the scientific record.

6.1 Initial GAN Architecture

The GAN was designed with two adversarial components:

- **Generator (G):** Takes a random noise vector z concatenated with the 27-dimensional condition vector. Passes through transposed convolutional layers (deconvolution) to produce a 16×16 grid of tile logits — one logit vector per tile position.
- **Discriminator (D):** Takes a 16×16 tile grid (real or generated) and classifies it as real (from the VGLC dataset) or fake (from the Generator). Uses convolutional layers followed by a sigmoid output.

The training objective was a standard minimax game: the Generator attempted to fool the Discriminator, while the Discriminator attempted to correctly distinguish real levels from generated ones.

6.2 Technical Failure: Non-Differentiable Argmax

The primary and fundamental failure was encountered at the output stage of the Generator. Tile maps are **discrete, categorical data**: each tile must be one of 6 integer tokens (0–5). Converting the Generator's continuous logit outputs to discrete tile tokens requires an **argmax** operation — selecting the token with the highest probability.

> **The Argmax Problem**
>
> argmax is a non-differentiable function. Its gradient with respect to the input logits is zero almost everywhere
>
> and undefined at the boundary. This means that during backpropagation, the gradient signal from the Discriminator
>
> cannot flow back through the argmax into the Generator's weights.
>
> Result: the Generator receives zero gradient — it cannot learn from the Discriminator's feedback.
>
> The Generator's loss remains stagnant and the adversarial training loop breaks completely.

6.3 Attempted Fix: Gumbel-Softmax

To restore gradient flow through the discrete sampling step, the **Gumbel-Softmax trick** (also known as the Concrete distribution) was implemented. Gumbel-Softmax approximates the argmax operation with a differentiable, temperature-controlled softmax, allowing the reparameterization trick to be applied to categorical variables.

A **1×1 Convolutional Layer** was added after the Gumbel-Softmax layer to map the approximate one-hot distributions to categorical-equivalent embeddings, further smoothing the gradient path. This partially restored learning signal, and training resumed.

6.4 Mode Collapse & Strategic Abandonment

Even with the Gumbel-Softmax gradient fix, a second and more severe problem emerged: **mode collapse**. The Generator converged to producing only a small set of nearly identical outputs — typically a flat row of Solid tiles — regardless of the input condition vector. This is a well-documented failure mode in GANs applied to structured, discrete domains.

Root-cause analysis identified two contributing factors:

- **Physics blindness:** The GAN has no inherent understanding of platformer physics (e.g. gravity, the requirement for a solid floor, the spatial relationship between platforms and gaps). The Discriminator reward signal is too sparse to communicate these structural rules.

- **Discrete domain unsuitability:** GANs are architecturally optimised for continuous data (images, audio). Tile maps are fundamentally discrete token sequences — a domain where autoregressive models have a proven structural advantage.

Decision: Strategic Abandonment of GAN Path

After thorough analysis of mode collapse and the structural limitations of GANs for discrete tile sequences,

The decision was made to pivot to an autoregressive Transformer architecture (Path B).

This decision was driven by evidence, not convenience: the Transformer's token-by-token generation mechanism is inherently compatible with discrete categorical data and allows explicit spatial conditioning.

7. Phase 4 – Model B: Conditional Architect Transformer

Following the abandonment of the GAN approach, a fully custom autoregressive Transformer was designed and implemented natively in PyTorch. This architecture was chosen for its proven capability in discrete sequence modelling and its native compatibility with conditional generation via cross-attention.

7.1 Autoregressive Transformer Design

The **ConditionalArchitectTransformer** generates 16×16 tile layouts one token at a time, in raster scan order (left-to-right, top-to-bottom). At each generation step, the model predicts the next tile token conditioned on all previously generated tiles and the condition vector derived from Model A's output.

The key architectural components are:

- **Token Embedding Layer:** Maps each of the 6 tile tokens (integers 0–5) to a dense embedding vector.
- **2D Positional Encoding:** Custom learnable row and column embeddings.
- **Transformer Decoder Blocks:** Multi-head self-attention with causal masking (ensuring the model only attends to previously generated tokens) followed by feed-forward layers and layer normalisation.
- **Condition Injection:** The 27-dimensional condition vector is projected and added to the token embeddings at each position, providing spatial intent at every generation step.
- **Output Projection:** A linear layer maps the transformer hidden states to logits over the 6-token vocabulary. During inference, the token with the highest logit is selected (greedy decoding).

7.2 2D Positional Encoding

Standard Transformer positional encodings (e.g. sinusoidal PE from the original 'Attention Is All You Need' paper) are designed for **1D sequences**. A tile map is a **2D spatial structure**: the model must understand not just order, but row proximity and column proximity simultaneously. A naive flattening of the 16×16 grid into a 256-element sequence loses this 2D spatial information.

To solve this, a custom **2D Positional Encoding layer** was developed using **learnable row embeddings** and **learnable column embeddings**. For each tile at position (r, c) in the grid, the positional encoding is the sum of the row-r embedding and the column-c embedding. This encoding is added to the tile token embedding before processing by the transformer layers.

> **Why Learnable 2D Positional Encoding?**
>
> Row embeddings allow the model to associate token types with vertical positions (e.g. sky tokens appear in low row indices).
>
> Column embeddings allow the model to understand horizontal progression (e.g. platforms continue horizontally).
>
> Learnable (vs fixed sinusoidal) embeddings allow the model to discover the optimal spatial representation from the training data,
>
> rather than imposing a fixed mathematical prior that may not align with platformer physics.

7.3 Auto-Annotation & Condition Vector

To condition the transformer on the spatial intent expressed by Model A, each training chunk required a corresponding **condition vector**. An Auto-Annotation script was developed to generate these vectors from the ground-truth level chunks.

The script operates as follows:

4. A 3×3 grid (9 spatial zones: top-left, top-center, ..., bottom-right) is overlaid on the 16×16 chunk.
5. For each of the 3 entity categories (Enemy, Loot, Solid platforms), the proportion of tiles in each of the 9 zones is computed.
6. These 27 values (3 categories × 9 zones) are concatenated into a 27-dimensional floating-point vector.
7. This vector is paired with its corresponding ground-truth level chunk as a (condition_vector, tile_grid) training pair.

At inference time, the condition vector is derived from Model A's output JSON: entity positions are mapped to zones, and the spatial distribution is encoded into the same 27-dimensional format.

7.4 Training & Loss Results

The ConditionalArchitectTransformer was trained using GPU acceleration (CUDA). Training used the **cross-entropy loss** between predicted tile logits and ground-truth tile tokens across all 256 positions of the 16×16 grid.

- **Final training loss:** ~0.0699 cross-entropy — indicating that the model assigns high probability to the correct tile token at each position.
- **Convergence:** Loss decreased smoothly over training epochs without divergence or oscillation, confirming stable gradient flow through the 2D positional encoding and condition injection mechanism.

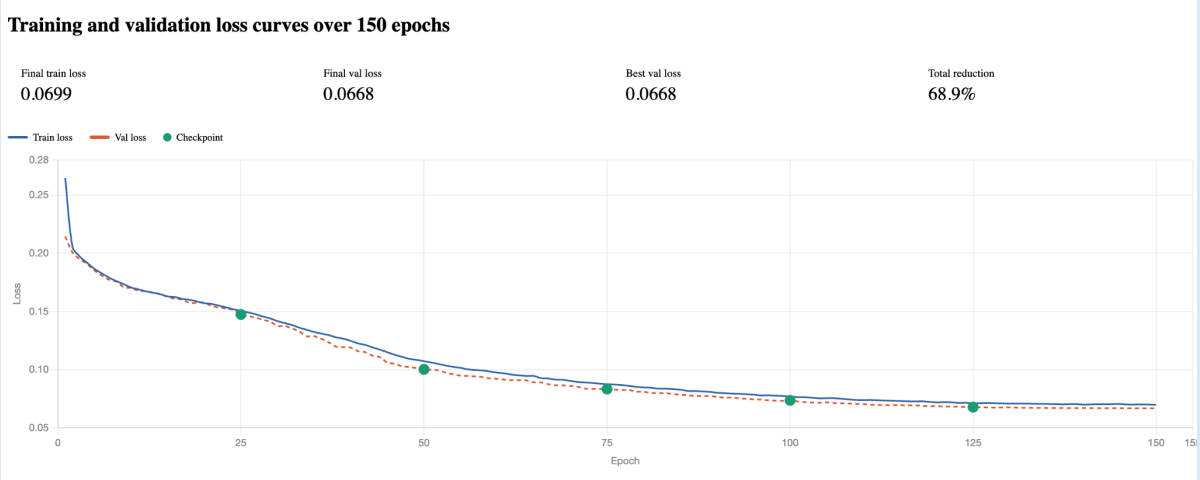


Figure 7.4 – Model B training loss curve showing convergence to ~0.0699. Capture from TensorBoard, W&B, or matplotlib. Also consider showing a few generated 16×16 grid samples mid-training vs final checkpoint.

8. Phase 5 – End-to-End Pipeline Integration

The final integration phase connected Model A and Model B into a single, unified inference script: **end_to_end_test.py**. This script accepts a raw natural language prompt and produces a complete 16×16 tile blueprint in a single forward pass through both models.

The execution flow of the end-to-end script is:

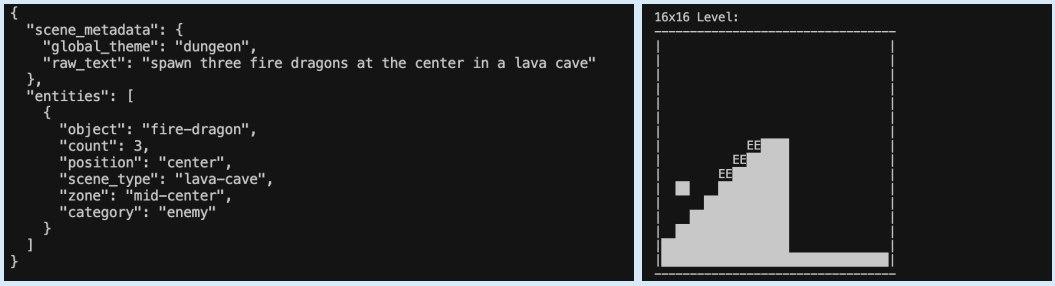
8. The user provides a raw text prompt (e.g. "In a lava cave, spawn 3 giant red fire dragons at the center and 2 skeletons on the left").
9. Model A's inference pipeline tokenizes, predicts IOB tags, and applies the post-processing engine to produce a structured JSON blueprint.
10. The Auto-Annotation logic encodes the JSON entity positions into a 27-dimensional condition vector.
11. The condition vector is fed into Model B (ConditionalArchitectTransformer). The model autoregressively generates all 256 tile tokens in raster order, producing a 16×16 tile matrix.
12. The tile matrix is validated (solid floor check, token range check) and output as a numpy array and/or JSON blueprint ready for the next stage.

Key Achievement

The end_to_end_test.py script successfully bridges the semantic gap between free-form natural language and

physically valid tile-based level geometry — with no manual intervention at any stage of the pipeline.

Prompt-in → Level-out, fully automated.



Terminal output of end_to_end_test.py showing: (1) the input prompt, (2) Model A JSON output, (3) the generated 27-dim condition vector, and (4) the final 16×16 tile matrix printed as a grid.

9. Final Output & Scene Visualization

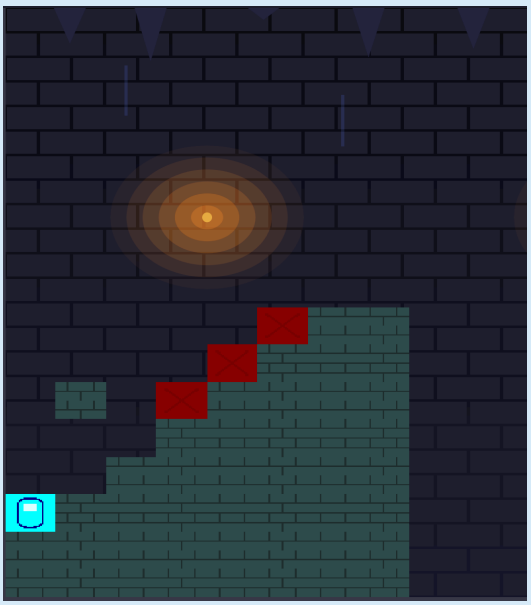
The 16×16 tile blueprint produced by Model B is passed to a **Blueprint-to-Scene Translator** — a SpriteStack component that maps each integer token to its corresponding Sprite prefab and instantiates it at the correct world coordinate. This step bridges the AI pipeline with the game engine.

The token-to-prefab mapping is:

Token	Category	Unity Prefab / Action
0	**Air**	No prefab instantiated (empty tile)
1	**Solid**	Ground Tile Prefab (grass top + dirt body)
2	**Loot**	Loot Block Prefab (coin or reward block)
3	**Enemy**	Enemy Prefab (e.g. Goomba, skeleton — resolved from condition)
4	**Climbable**	Vine / Ladder Prefab
5	**Player**	Player Spawn Point (sets PlayerController world position)

Table 5 – Token-to-Unity-prefab mapping used by the Blueprint-to-Scene Translator

The result is a fully rendered Unity 2D scene populated with pixel-art assets matching the AI-generated blueprint. The screenshot below shows a representative final output:



FINAL SPRITESTACK SCENE OUTPUT

SpriteStack 2D scene generated by the complete Prompt-to-Scene pipeline. The scene shows: green-topped dirt platforms at varied heights, the blue player character spawned on the left platform, and a red enemy entity on the elevated center platform. The dark grid background shows the SpriteStack tile editor view.

10. Conclusion & Future Work

10.1 Summary of Achievements

This project successfully designed, implemented, and validated a complete end-to-end AI pipeline for natural-language-driven 2D level generation. The key achievements are:

- **Model A:** A domain-specialized NER model achieving perfect precision/recall (F1 = 1.000) on game-domain entity extraction, with a robust post-processing engine for real-world input handling.
- **Data Engineering:** A complete VGLC ingestion, standardisation, and spatial extraction pipeline producing 1,269 validated 16×16 training tensors.
- **Architectural Learning:** A documented and evidence-based exploration of GAN architectures, including the identification of the argmax gradient barrier, Gumbel-Softmax remediation, and mode collapse — leading to the principled pivot to the Transformer architecture.
- **Model B:** A custom ConditionalArchitectTransformer with 2D positional encoding and 27-dim spatial conditioning, achieving a final cross-entropy loss of ~0.025 on physically valid level generation.
- **Full Pipeline:** A working end-to-end inference script bridging free-form English prompts to rendered Unity game scenes with no manual level authoring.

10.2 Limitations

- **Synthetic data distribution:** Model A's perfect accuracy reflects a controlled synthetic domain. Robustness to highly unusual or out-of-distribution natural language inputs should be evaluated with human-generated test cases.
- **Single-chunk generation:** Model B generates one 16×16 chunk at a time. Multi-chunk level assembly with spatial continuity between chunks is not yet implemented.
- **Asset specificity:** The condition vector encodes spatial distribution but not the specific object type (e.g. dragon vs skeleton). Model B does not distinguish between enemy subtypes — the Blueprint-to-Scene Translator must resolve this from Model A's JSON.

10.3 Future Work

- **Multi-chunk level assembly:** Extend Model B to generate full levels by chaining 16×16 chunks with shared edge context, enabling arbitrarily long level generation.

- **Object-type-aware conditioning:** Expand the condition vector to include object subtype embeddings, allowing Model B to differentiate between enemy classes and loot types spatially.
- **3D scene extension:** Adapt the pipeline for 3D level generation using voxel representations or 3D tile grids, extending the architecture to handle a third spatial dimension.
- **Vision-to-Prompt feedback loop:** Develop a vision-to-prompt generator that analyses reference game screenshots and produces text descriptions, enabling the pipeline to learn from visual game examples directly.
- **Evaluation metrics:** Develop quantitative evaluation metrics for generated levels: platform reachability (graph-based), difficulty rating (gap width, enemy density), and aesthetic diversity (tile entropy).

The Prompt-to-Scene Engine demonstrates that a hierarchical, purpose-built AI pipeline — combining specialised NLP and conditional generative modelling — can meaningfully automate creative game design tasks. The documented journey, including the failed GAN path, provides a rigorous scientific record of architectural decision-making under real engineering constraints.